
Área Científica	Programação de Sistemas de Informação
Unidade Curricular	Introdução às Linguagens e Tecnologias Web
Ano Letivo / Semestre	2025/2026 – 2º Semestre
Docente Responsável	Prof. ^a Catarina Silva
Documento	Guião Prático Laboratorial - jQuery
Tema	<i>Bibliotecas JavaScript: jQuery, Manipulação e Efeitos</i>

Objetivos do Guião Prático

Este guião prático foca-se na transição para a utilização da biblioteca jQuery como ferramenta de simplificação do desenvolvimento front-end. No final desta sessão, deverá ser capaz de manipular a interface Web com maior agilidade e menos linhas de código.

- Integrar o jQuery via CDN e localmente.
- Utilizar o motor de seleção Sizzle (seletores complexos).
- Aplicar técnicas de navegação na árvore DOM (Traversal).
- Implementar lógica de eventos e animações de forma declarativa.
- Efetuar pedidos de rede assíncronos (AJAX) simplificados.

Mini Projeto para Consolidação

Fase 1: Investigação e Arquitetura

O objetivo aqui é planear antes de escrever. Sigam estes passos:

1. **Exploração da API:** Abram o browser e visitem `jsonplaceholder.typicode.com/users`. Observem que cada utilizador tem um id. Este ID será a "chave" para procurar os posts desse utilizador em `/posts?userId=X`.
2. **Esboço HTML:** No vosso `index.html`, criem dois contentores principais: uma div `id="lista-utilizadores"` e uma div `id="painel-conteudo"`.

3. **Decisão de Estilo:** Criem uma classe CSS simples para o "Card" do utilizador (ex: borda, padding, sombra) para que saibam como injetar o HTML mais tarde.

Fase 2: Implementação Estratégica

Sigam esta sequência lógica para construir a aplicação:

1. **O Pedido Inicial:** No `main.js`, usem `$.get("../users", function(data) { ... })`.
2. **Renderização de Cards:** Dentro do sucesso do pedido, usem o ciclo `$.each(data, function(i, user) { ... })`. Para cada utilizador, façam um `append` ao conteúdo da lista com este formato:

```
<div class="card" data-id="1">
  <h3>Nome do Utilizador</h3>
  <button class="btn-ver-posts">Ver Posts</button>
</div>
```

3. **Escuta de Eventos (Delegação):** Usem o seletor do pai para ouvir os botões que acabaram de criar: `$("#lista-utilizadores").on("click", ".btn-ver-posts", function() { ... })`.
4. **Captura do ID e Novo Pedido:** Dentro do clique, usem `let id = $(this).closest(".card")`. Façam um novo `$.get` para buscar os posts desse `userId`.
5. **Exibição de Posts:** Limpem o painel lateral com `.empty()` e injetem os títulos dos posts recebidos.

Fase 3: Otimização e Modernização

Agora, "limpem" e modernizem o vosso código:

1. **Implementar Chaining:** Verifiquem onde estão a manipular o mesmo elemento várias vezes. Juntem tudo: `$("#msg").text("Sucesso").css("color", "green").fadeIn();`
2. **Feedback Visual (Efeitos):** Adicionem um `.slideUp()` ao painel de posts antes de o limparem, e um `.slideDown()` após injetarem o novo conteúdo para uma transição suave.
3. **Desafio Alpine.js:** Escolham um botão simples (ex: "Mostrar/Esconder Sidebar"). No HTML, adicionem `x-data=" show: true "` e usem `x-show="show"` no painel para verem como a reatividade funciona sem precisar de seletores no JS.

Fase 4: Apresentação

Preparem-se para mostrar aos colegas:

- Mostrem como a vossa aplicação carrega os dados "em cascata" (Utilizador → Posts).
- Expliquem como o `closest()` ajudou a encontrar o ID correto do card clicado.
- Partilhem uma coisa que o Alpine.js ou o Vanilla JS fez de forma mais simples (ou mais difícil) que o jQuery.